

XGBoost for Credit Card Fraud Detection

Concepts, workflow, and evaluation considerations

1. Problem Overview

Credit card fraud detection is a highly imbalanced binary-classification problem. A large majority of transactions are legitimate, while fraudulent transactions form a much smaller class.

2. Why XGBoost?

XGBoost is a gradient-boosted decision-tree algorithm. It builds an ensemble of trees sequentially, with later trees focusing on errors made by earlier trees. It is widely used for structured and tabular data.

3. Handling Class Imbalance

When fraud cases are rare, a model can appear accurate while failing to detect fraud. Class weighting, threshold tuning, stratified evaluation, and appropriate metrics can help address this problem. The `scale_pos_weight` parameter is one option for weighting the positive class.

4. Important Metrics

Precision measures how many transactions predicted as fraud are actually fraudulent. **Recall** measures how many actual fraud cases are detected. **F1-score** combines precision and recall. **ROC-AUC** measures ranking performance across thresholds, while **PR-AUC** is particularly informative for highly imbalanced classification.

5. Example Evaluation

An example model evaluation can report ROC-AUC of 0.9826, PR-AUC of 0.8795, precision of 0.86, recall of 0.85, and F1-score of 0.86. These numbers should always be interpreted together with the test-set construction, class distribution, and decision threshold.

6. Deployment Pipeline

A practical system can expose the trained model through a Python API such as FastAPI. A frontend can submit transaction features to the API and display the predicted class and model confidence. Production systems should also include authentication, validation, logging, monitoring, and model-version management.

This document is provided as downloadable project/reference material.